ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO CUỐI KÌ

Thực tập đồ án môn học đa ngành - hướng hệ thống thông tin

Đề tài: Hệ thống tưới tiêu cho cây trồng

Giảng viên hướng dẫn:	Nguyễn Quang Hùng	
Lớp:	L01	
Sinh viên thực hiện:	Trần Hoàng Phúc Đạt	2210718
	Võ Hoàng Huy	2211299
	Phạm Gia Lương	2211960
	Trần Minh Quang	2212757
	Võ Thị Ánh Tuyết	2213826
	Thái Huy Hoàng	2211116

Thành phố Hồ Chí Minh, Tháng 04 Năm 2025

Mục lục

1 Giới thiệu **2**

2 Cơ sở lý thuyết **2**
2.1 Khái niệm Internet of Things (IoT) trong nông nghiệp 2
2.2 Hệ thống tưới tiêu truyền thống vs tưới tiêu thông minh 2
2.3 Các thành phần chính của hệ thống tưới tiêu thông minh cho cây trồng 3
2.3.a Giới thiệu về NodeMCU ESP32 3
2.3.b Các thông số kỹ thuật chính 3
2.4 Các thành phần phụ của hệ thống 4
2.4.a Cảm biến độ ẩm đất 4
2.4.b Cảm biến nhiệt độ, độ ẩm DHT11 5
2.4.c Bơm và Relay 5
2.4.d Màn hình LCD 16x2 6
2.5 Các giao thức kết nối được sử dụng 7

3 Phân tích và triển khai mô hình **7**
3.1 Tổng quan về hệ thống 7
3.2 Use-case Diagram 8
3.3 Kết nối phần cứng 9
3.4 Lập trình điều khiển 9
3.5 Học máy 13
3.6 Dashboard 18
3.7 Web 19

4 Hình ảnh thực tế **20**

5 Kết quả thực nghiệm và đánh giá **20**
5.1 Kết quả 20
5.2 Đánh giá hiệu quả 20

6 Lời cảm ơn **22**

TÀI LIỆU THAM KHẢO **23**

1 Giới thiệu

Trong bối cảnh biến đổi khí hậu ngày càng nghiêm trọng cùng với sự gia tăng về nhu cầu sử dụng nước trong nông nghiệp, việc xây dựng các giải pháp tưới tiêu hiệu quả, tiết kiệm và bền vững đang trở thành một yêu cầu cấp thiết. Nông nghiệp vẫn là lĩnh vực tiêu thụ nước lớn nhất hiện nay, do đó việc tối ưu hóa hệ thống tưới tiêu không chỉ góp phần nâng cao năng suất cây trồng mà còn đóng vai trò quan trọng trong bảo vệ tài nguyên nước và môi trường sống.

Đề tài "Hệ thống tưới tiêu cho cây trồng ứng dụng công nghệ Internet of Things (IoT)" được thực hiện nhằm nghiên cứu, thiết kế và triển khai một mô hình tưới thông minh có khả năng hoạt động tự động và giám sát từ xa. Hệ thống sử dụng các cảm biến để thu thập dữ liệu môi trường như độ ẩm đất, nhiệt độ không khí, cường độ ánh sáng,... Sau đó, các dữ liệu này sẽ được xử lý để đưa ra quyết định tưới hợp lý thông qua vi điều khiển và được điều khiển từ xa thông qua nền tảng IoT.

So với báo cáo giữa kỳ – vốn tập trung vào nghiên cứu lý thuyết, khảo sát các công nghệ, lựa chọn phần cứng và thiết kế hệ thống – báo cáo cuối kỳ này sẽ trình bày chi tiết quá trình xây dựng mô hình thực tế, mô phỏng, lập trình điều khiển, kết nối với nền tảng IoT (ThingsBoard), cũng như đánh giá hoạt động của hệ thống qua các kịch bản mô phỏng và thực nghiệm.

Thông qua đồ án này, nhóm thực hiện mong muốn góp phần vào xu hướng chuyển đổi số trong nông nghiệp, đồng thời tạo nền tảng để phát triển các hệ thống canh tác thông minh, tiết kiệm tài nguyên và thân thiện với môi trường.

2 Cơ sở lý thuyết

2.1 Khái niệm Internet of Things (IoT) trong nông nghiệp

IoT trong nông nghiệp là các thiết bị thông minh, cảm biến được nối với điều khiển tự động trong suốt quá trình vận sản xuất canh tác góp phần tránh bị biến đổi khí hậu và hiệu ứng nhà kính.

Thêm vào đó, ứng dụng IoT trong nông nghiệp giúp nâng cao sản lượng và chất lượng sản phẩm, giảm sử dụng tài nguyên và chi phí, tối ưu hóa quy trình sản xuất. Đồng thời, đảm bảo được sức khỏe cho cộng đồng

2.2 Hệ thống tưới tiêu truyền thống vs tưới tiêu thông minh

Tưới cây truyền thống yêu cầu sự can thiệp bởi phương pháp thủ công để bật và tắt vòi nước. Và người dùng phải di chuyên dụng cụ tưới đi tưới khắp khu vườn của mình. Trong khi đó, với hệ thống tưới tự động được thiết kế nhằm giúp các hoạt động tưới cây diễn ra theo quy trình tự động, với hệ thống hẹn giờ sẵn, các cây trồng sẽ được tưới đúng thời gian quy định và lượng nước tưới cũng được phân bố đều.

2.3 Các thành phần chính của hệ thống tưới tiêu thông minh cho cây trồng

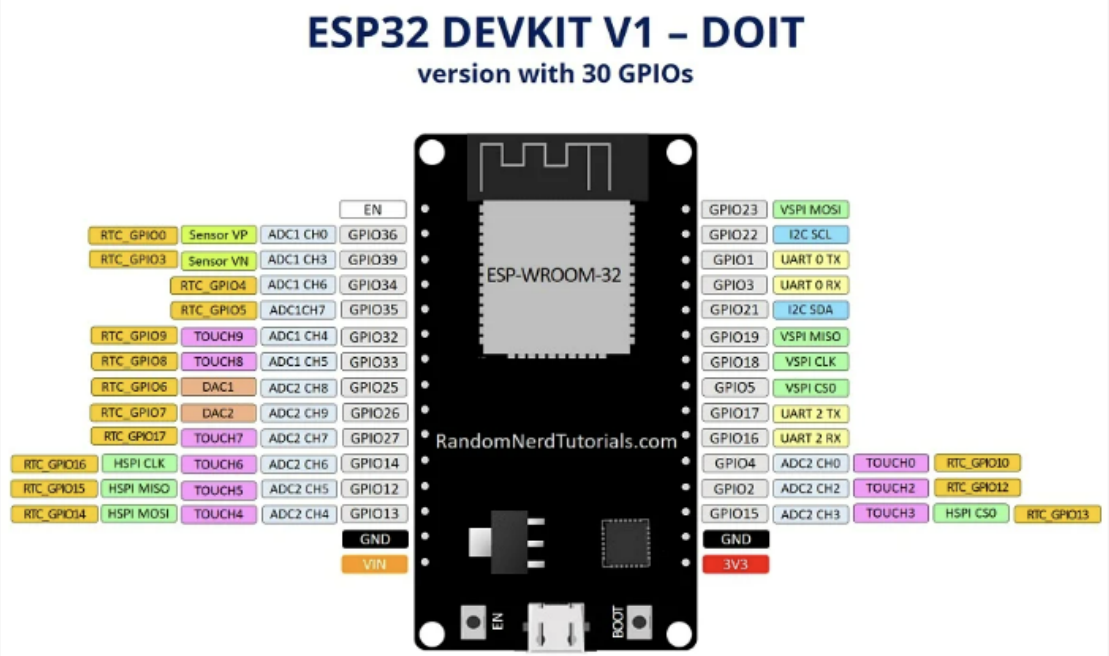
2.3.a Giới thiệu về NodeMCU ESP32

ESP32 là một module Wi-Fi với khả năng kết nối Internet và được tích hợp sẵn trên một số board nhúng như NodeMCU, Wemos, và ESP-01. ESP32 có thể hoạt động như một điểm truy cập (access point), một client kết nối đến một điểm truy cập khác, hoặc cả hai đều được. Nó được sử dụng rộng rãi trong các ứng dụng IoT (Internet of Things) như cảm biến thông minh, hệ thống kiểm soát thiết bị, hoặc các ứng dụng điều khiển từ xa. Module này có giá thành rẻ và rất dễ sử dụng, cùng với đó là khả năng tương thích với nhiều loại vi điều khiển khác nhau.
ESP32 NodeMCU là một nền tảng IoT mã nguồn mở, được phát triển bởi một nhóm kỹ sư tại Trung Quốc. Nền tảng này được xây dựng trên ESP32, một vi điều khiển Wi-Fi SoC (System on a Chip) được sản xuất bởi Espressif Systems. NodeMCU cung cấp một bộ SDK để lập trình cho ESP32 bằng ngôn ngữ Lua hoặc C++. Với các tính năng như Wi-Fi, GPIO, ADC, I2C, SPI, PWM và một số tính năng khác, NodeMCU ESP32 được sử dụng rộng rãi trong các ứng dụng IoT như kiểm soát thiết bị, thu thập dữ liệu và giao tiếp với các thiết bị khác.

2.3.b Các thông số kỹ thuật chính

Thông số	**Mô tả**
Microcontroller	ESP32
Điện áp hoạt động	3.3V DC
Số chân I/O	17 chân GPIO
Kết nối mạng	**WiFi** 802.11 b/g/n
Giao diện mạng	TCP/IP
Đồng hồ thời gian thực (RTC)	Không tích hợp
Bộ nhớ trong	4MB
RAM	80KB
Cổng nạp	Micro-USB
Hỗ trợ các giao thức	**MQTT**, **CoAP**, HTTP/HTTPS
Kích thước	49 × 24.5 × 13mm

Bảng 1: *Thông số kỹ thuật của NodeMCU ESP32*

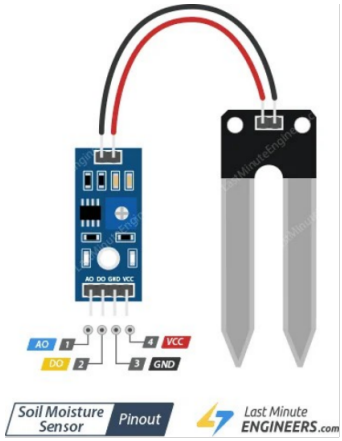


Hình 1: *Sơ đồ các chân của Node MCU*

2.4 Các thành phần phụ của hệ thống

2.4.a Cảm biến độ ẩm đất

Nguyên lý cảm biến độ ẩm đất dựa trên sự tương tác giữa đất và điện trở. Khi đất ẩm, nước trong đất sẽ làm tăng khả năng dẫn điện của đất. Cảm biến độ ẩm đất sử dụng hai đầu cảm biến hoặc điện cực để đo điện trở của môi trường đất. Cảm biến thường được làm từ hai đầu kim loại không rỉ như thép không gỉ hoặc đồng. Hai đầu kim loại này được chìm vào đất để tiếp xúc với đất xung quanh. Khi đất ẩm, nước trong đất sẽ tạo thành một lớp dẫn điện xung quanh các đầu kim loại, làm tăng khả năng dẫn điện của môi trường. Cảm biến sẽ đo điện trở giữa hai đầu kim loại để xác định mức độ ẩm của đất. Khi đất khô, điện trở sẽ cao vì không có nước để tạo dẫn điện. Ngược lại, khi đất ẩm, điện trở sẽ thấp hơn do sự dẫn điện của nước trong đất.



Hình 2: *Sơ đồ chân cảm biến độ ẩm đất*

2.4.b Cảm biến nhiệt độ, độ ẩm DHT11

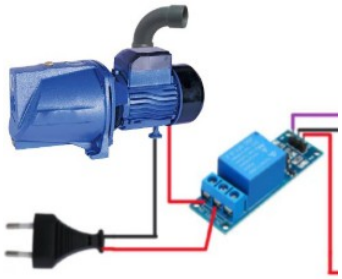
Cảm biến độ ẩm và nhiệt độ DHT11 là cảm biến rất thông dụng hiện nay vì chi phí rẻ và rất dễ lấy dữ liệu thông qua chuẩn giao tiếp 1 wire.
Chuẩn giao tiếp 1 wire là dùng 1 chân Digital để truyền dữ liệu. Bộ tiền xử lý tín hiệu được tích hợp trong cảm biến giúp bạn có thể đọc dữ liệu chính xác mà không phải qua bất kỳ tính toán nào.



Hình 3: *Module cảm biến nhiệt độ, độ ẩm DHT11.*

2.4.c Bơm và Relay

Bơm là một linh kiện không thể thiếu trong hệ thống tưới cây, với các hệ thống có quy mô khác nhau, kích thước của bơm sẽ thay đổi để phù hợp với điều kiện thực tế nhưng đối với các hệ thống cỡ nhỏ thì chỉ cần sử dụng bơm cỡ nhỏ và để điều khiển bơm này cần một Relay 5V. Relay 5V là linh kiện điện tử cho phép ESP32 điều khiển các thiết bị cao áp như bơm nước một cách an toàn. Khi ESP32 gửi tín hiệu điều khiển, relay sẽ bật hoặc tắt bơm, đảm bảo việc cung cấp nước được tự động hóa. Relay bảo vệ vi điều khiển khỏi tải điện áp cao và giúp hệ thống vận hành ổn định, tăng độ bền và an toàn cho toàn bộ hệ thống tưới cây thông minh.



Hình 4: *Hình ảnh Bơm và Relay được sử dụng trong các hệ thống cỡ nhỏ*

2.4.d Màn hình LCD 16x2

Hiển thị các thông số trực tiếp và liên tục qua màn hình LCD 16x2 giúp dễ dàng quan sát tình hình của cây trồng cũng như kiểm tra xem các thiết bị nào trong hệ thống đang tắt/mở. Và để làm được điều này cần một màn hình LCD mà như trong hệ thống này là màn hình LCD 16x2.

Thông số	**Mô tả**
Điện áp hoạt động	5V
Kích thước	98 × 60 × 13.5 mm
Màu sắc	Chữ trắng, nền xanh dương hoặc chữ đen nền xanh lá
Khoảng cách giữa hai chân kết nối	0.1 inch, tiện dụng khi kết nối với Breadboard
Tên các chân	Được ghi ở mặt sau của màn hình LCD hỗ trợ việc kết nối và đi dây điện
Đèn nền	Có đèn LED nền, có thể dùng biến trở hoặc PWM để điều chỉnh độ sáng nhằm sử dụng ít điện năng hơn
Số dây tín hiệu điều khiển	6 dây tín hiệu
Bộ ký tự hỗ trợ	Bộ ký tự được xây dựng hỗ trợ tiếng Anh và tiếng Nhật, xem thêm HD44780

Bảng 2: *Thông số kỹ thuật của màn hình LCD 16x2*



Hình 5: *Màn hình LCD 16*2*

2.5 Các giao thức kết nối được sử dụng

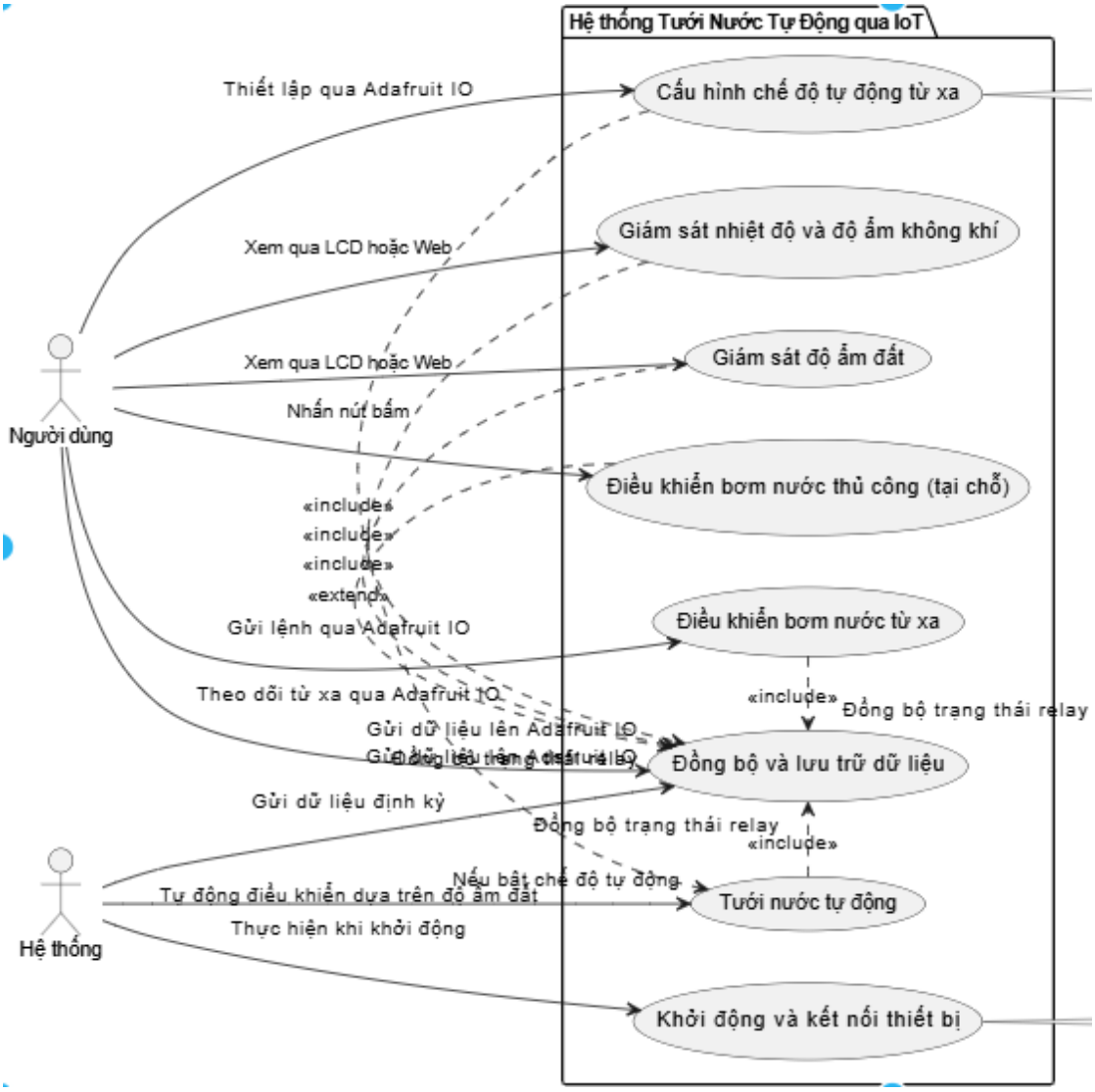
- **Wi-Fi:** ESP32 sử dụng Wi-Fi để kết nối với mạng và nền tảng Blynk.
- **I2C:** Kết nối giữa ESP32 và màn hình LCD.
- **GPIO:** Điều khiển relay và đọc trạng thái cảm biến.

3 Phân tích và triển khai mô hình

3.1 Tổng quan về hệ thống

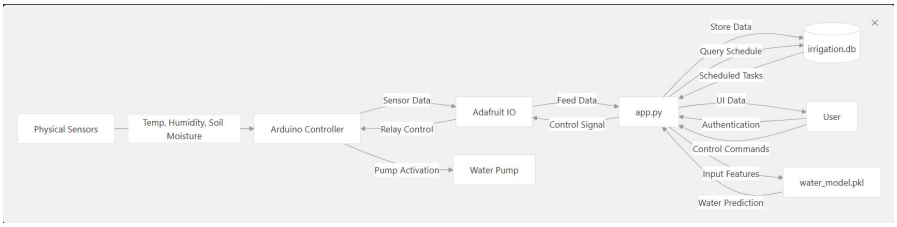
Hệ thống chăm sóc cây thông minh sẽ thu thập các thông số của môi trường như độ ẩm đất, nhiệt độ môi trường và xử lí, khi môi trường có sự thay đổi lớn (nhiệt độ quá cao, độ ẩm quá thấp). Hệ thống sẽ tiến hành can thiệp (mở máy bơm nước,...) để cân bằng lại nhiệt độ và độ ẩm.

3.2 Use-case Diagram



Hình 6: *Use-case Diagram*

3.3 Kết nối phần cứng



Hình 7: *Sơ đồ kết nối phần cứng*

3.4 Lập trình điều khiển

- Kết nối Wi-Fi:

```
const char* ssid = "abcd";
const char* password = "12345678";

void setup() {
  ...
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.println("Connecting to WiFi...");
  }
  Serial.println("Connected to WiFi");
  ... }

```

- Gửi dữ liệu MQTT lên Adafruit IO:

```
void sendSensorData() {
  float h = dht.readHumidity();
  float t = dht.readTemperature();
  int soilValue = analogRead(soil);
  soilValue = map(soilValue, 0, 4095, 0, 100);
  soilValue = (soilValue - 100) * -1;

  MQTT_connect();
  temperatureFeed.publish(t);
  humidityFeed.publish(h);
  soilMoistureFeed.publish((int32_t)soilValue);
  Serial.println("Data sent to Adafruit IO:");
  Serial.print("Temperature: ");
  Serial.print(t);
  Serial.print(" C, Humidity: ");
  Serial.print(h);
  Serial.print(" %, Soil Moisture: ");
  Serial.print(soilValue);
```

```
  Serial.println(" %");
}

```

- Đọc cảm biến:
  - Đọc dữ liệu từ cảm biến DHT11 (nhiệt độ và độ ẩm):

```
void DHT11sensor() {
  float h = dht.readHumidity();
  float t = dht.readTemperature();
  if (isnan(h) || isnan(t)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
  }
  lcd.setCursor(0, 0);
  lcd.print("T:");
  lcd.print(t, 1);
  lcd.setCursor(8, 0);
  lcd.print("H:");
  lcd.print(h, 1);
  Serial.print("Temperature: ");
  Serial.print(t);
  Serial.print(" C, Humidity: ");
  Serial.print(h);
  Serial.println(" %");
}

```

  - Đọc dữ liệu từ cảm biến độ ẩm đất:

```
void soilMoistureSensor() {
  int value = analogRead(soil);
  value = map(value, 0, 4095, 0, 100);
  value = (value - 100) * -1;
  lcd.setCursor(0, 1);
  lcd.print("S:");
  lcd.print(value);
  lcd.print(" ");

  if (autoEnable) {
    unsigned long currentMillis = millis();
    if (value < soilMin && relay1State == LOW) {
      relay1State = HIGH;
      digitalWrite(RELAY_PIN_1, relay1State);
      if (currentMillis - previousRelaySendMillis >= relaySendCooldown) {
        MQTT_connect();
        relayFeed.publish("ON");
        previousRelaySendMillis = currentMillis;
      }
      lcd.setCursor(11, 1);
      lcd.print("W:ON ");
    } else if (value >= soilMax && relay1State == HIGH) {
      relay1State = LOW;
```

```
    digitalWrite(RELAY_PIN_1, relay1State);
    if (currentMillis - previousRelaySendMillis >= relaySendCooldown) {
      MQTT_connect();
      relayFeed.publish("OFF");
      previousRelaySendMillis = currentMillis;
    }
    lcd.setCursor(11, 1);
    lcd.print("W:OFF");
  }
 }
}
```

- Điều khiển relay:

  - Điều khiển relay dựa trên lệnh từ MQTT.

```
void relayCallback(char *data, uint16_t len) {
  String message = String(data);
  Serial.print("Received from relaycontrol: ");
  Serial.println(message);

  if (message == "ON") {
    relay1State = HIGH;
  } else if (message == "OFF") {
    relay1State = LOW;
  }
  digitalWrite(RELAY_PIN_1, relay1State);
  updateLCDForRelay();

  unsigned long currentMillis = millis();
  if (currentMillis - previousRelaySendMillis >= relaySendCooldown) {
    MQTT_connect();
    relayFeed.publish(relay1State == HIGH ? "ON" : "OFF");
    previousRelaySendMillis = currentMillis;
  }}
```

  - Điều khiển relay thông qua nút nhấn vật lý:

```
void checkPhysicalButton() {
  if (digitalRead(PUSH_BUTTON_1) == LOW) {
    delay(200);
    relay1State = !relay1State;
    digitalWrite(RELAY_PIN_1, relay1State);

    unsigned long currentMillis = millis();
    if (currentMillis - previousRelaySendMillis >= relaySendCooldown) {
      MQTT_connect();
      relayFeed.publish(relay1State == HIGH ? "ON" : "OFF");
      previousRelaySendMillis = currentMillis;
      Serial.println(relay1State == HIGH ? "p: ON" : "p: OFF");
    }
    while (digitalRead(PUSH_BUTTON_1) == LOW);
    delay(200);
  }
```

```
}
```

- Hiển thị thông tin trên LCD:
  - Hiển thị nhiệt độ, độ ẩm, độ ẩm đất, trạng thái bơm, và chế độ tự động trên màn hình LCD.

```
void updateLCDForRelay() {
  if (relay1State == HIGH) {
    lcd.setCursor(11, 1);
    lcd.print("W:ON ");
  } else {
    lcd.setCursor(11, 1);
    lcd.print("W:OFF");
  }
}

void updateLCDForAuto() {
  lcd.setCursor(5, 1);
  lcd.print("A:");
  lcd.print(autoEnable ? "ON " : "OFF");
}
```

- Điều khiển thủ công
  - Gửi lệnh bật/tắt bơm thông qua giao diện web:

```
@app.route('/control', methods=['POST'])
@login_required
def control():
    feed = request.form['feed']
    value = request.form['value']
    success = send_feed_data(feed, value)
    print(f"Control request: feed={feed}, value={value}, success={success}")
    return jsonify({'success': success})
```

- Lập lịch bật/tắt bơm
  - Lập lịch tự động bật/tắt bơm tại các thời điểm cụ thể:

```
@app.route('/schedule', methods=['POST'])
@login_required
def schedule():
    try:
        scheduled_time_str = request.form['scheduled_time']
        off_time_str = request.form['off_time']
        scheduled_time = datetime.strptime(scheduled_time_str, '%Y-%m-%dT%H:%M')
        off_time = datetime.strptime(off_time_str, '%Y-%m-%dT%H:%M')

        if off_time <= scheduled_time:
            print("Li: Thi gian tt phi sau thi gian bt")
            return jsonify({'error': 'Thi gian tt phi sau thi gian bt'})

        new_schedule = Schedule(scheduled_time=scheduled_time, off_time=off_time)
```

```
        db.session.add(new_schedule)
        db.session.commit()
        print(f" lp lch: Bt lc {scheduled_time}, Tt lc {off_time}")
        return jsonify({'success': True})
    except ValueError as e:
        print("Li d liu lch trong schedule:", str(e))
        return jsonify({'error': 'D liu lch khng hp l'})
    except Exception as e:
        print("Li trong schedule:", str(e))
        return jsonify({'error': str(e)})
```

3.5 Học máy

- Tải và tiền xử lý dữ liệu:
  - Đọc dữ liệu: Dữ liệu được đọc từ file CSV *data_core.csv* vào *DataFrame* bằng thư viện *Pandas*
  - Xử lý giá trị thiếu: Các hàng chứa giá trị thiếu (NaN) bị loại bỏ bằng phương thức *dropna()* để đảm bảo tính toàn vẹn của dữ liệu.

```
df = pd.read_csv("data_core.csv")

df.dropna(inplace=True)
```

- Mã hóa dữ liệu phân loại:
  - Label Encoding: Các cột dữ liệu phân loại *SoilType, CropType* được chuyển đổi thành số nguyên bằng *LabelEncoder* để phù hợp với đầu vào của mô hình.
  - Lưu bộ mã hóa: Hai bộ mã hóa được lưu vào file *encoder_soil.pkl, encoder_crop.pkl* để sử dụng lại khi dự đoán trên dữ liệu mới.

```
encoder_soil = LabelEncoder()
encoder_crop = LabelEncoder()
df['SoilType'] = encoder_soil.fit_transform(df['SoilType'])
df['CropType'] = encoder_crop.fit_transform(df['CropType'])

joblib.dump(encoder_soil, "encoder_soil.pkl")
joblib.dump(encoder_crop, "encoder_crop.pkl")
```

- Chuẩn hóa dữ liệu số:
  - Standard Scaling: Các đặc trưng số *Temperature, Humidity, Moisture* được chuẩn hóa về cùng tỷ lệ (mean=0, std=1) bằng *StandardScaler* để tránh ảnh hưởng của đơn vị đo lên mô hình.
  - Lưu bộ chuẩn hóa: Bộ *scaler* được lưu vào file *scaler.pkl* để áp dụng cùng quy trình cho dữ liệu mới.

```
scaler = StandardScaler()
df[['Temparature', 'Humidity', 'Moisture']] = scaler.fit_transform(
    df[['Temparature', 'Humidity', 'Moisture']]
)
```

```

joblib.dump(scaler, "scaler.pkl")
```

- Chia dữ liệu và huấn luyện mô hình:
  - Tách biến đầu vào/đầu ra:
  + **X:** Chứa các đặc trưng đầu vào (đã loại bỏ cột mục tiêu và các cột không cần thiết như *Nitrogen, Potassium, Phosphorous, FertilizerName*).
  + **Y:** Chứa nhãn cần dự đoán (WaterNeeded).

  - Chia train/test: Dữ liệu được chia thành tập huấn luyện (80%) và tập kiểm tra (20%) với *random_state = 42* để đảm bảo tái lập kết quả.
  - Huấn luyện bằng Random Forest Regression: Mô hình RandomForestRegressor với 100 cây quyết định (cây quyết định được xây dựng dựa trên các cột dữ liệu trong tập X, mỗi cây sẽ có 1 chiến lược xây dựng cây khác nhau) được huấn luyện trên tập train. Dữ liệu mẫu được đưa vào 100 cây quyết định, mỗi cây sẽ cho ra 1 kết quả dự đoán khác nhau, kết quả cuối cùng là giá trị trung bình của 100 cây đó. Random Forest phù hợp cho bài toán hồi quy do khả năng xử lý dữ liệu phi tuyến và giảm overfitting.

```
X = df.drop(columns=['Nitrogen', 'Potassium', 'Phosphorous', 'WaterNeeded', 'FertilizerName'])
y = df['WaterNeeded']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- Thực hiện chuẩn hóa data rồi huấn luyện 4 mô hình và kiểm tra độ chính xác:
  - Mô hình 1 - Random Forest:

```
        model_rf = RandomForestRegressor(n_estimators=100, random_state=42)
        model_rf.fit(X_train, y_train)
        ......
        score = model_rf.score(X_test, y_test)
        print(f"RF Model Score: {score:.2f}")
        joblib.dump(model_rf, "models/model_rf.pkl")

```

  - Mô hình 2 - Linear Regression:

```
        model_lr = LinearRegression()
        model_lr.fit(X_train, y_train)
        ......
        score = model_lr.score(X_test, y_test)
        print(f"LR Model Score: {score:.2f}")
        joblib.dump(model_lr, "models/model_lr.pkl")

```

  - Mô hình 3 - XGBoost:

```
        model_xgb = xgb.XGBRegressor()
        model_xgb.fit(X_train, y_train)
        ......
        score = model_xgb.score(X_test, y_test)
        print(f"XGB Model Score: {score:.2f}")
        joblib.dump(model_xgb, "models/model_xgb.pkl")

```

- Mô hình 4 - MLP:

```
        model_mlp = MLPRegressor(max_iter=7000)
        model_mlp.fit(X_train, y_train)
        .......
        score = model_mlp.score(X_test, y_test)
        print(f"MLP Model Score: {score:.2f}")
        joblib.dump(model_mlp, "models/model_mlp.pkl")

```

- Cung cấp api để thực hiện dự đoán, cập nhật điểm tin cậy cho các model
  - Điểm độ tin cậy ban đầu cho từng mô hình:

```
    self.model_scores = {
        "RandomForest": 0.98,
        "LinearRegression": 0.75,
        "XGBoost": 0.99,
        "MLP": 0.89
    }
        def predict_all(self, input_data):
    """Predict from all models"""
    preds = {
        "RandomForest": self.rf_model.predict(input_data)[0],
        "LinearRegression": self.lr_model.predict(input_data)[0],
        "XGBoost": self.xgb_model.predict(input_data)[0],
        "MLP": self.mlp_model.predict(input_data)[0]
    }

```

- Làm tròn:

```
    preds = {model: round(value, 4) for model, value in preds.items()}
    return preds

def predict_best(self, input_data):
    """Return the weighted average predicted value"""
    preds = self.predict_all(input_data)
    total_score = sum(self.model_scores.values())
    weighted_sum = sum(preds[model] * self.model_scores[model] for model in preds)
    weighted_avg = weighted_sum / total_score
    .....
    weighted_avg = round(weighted_avg, 4)
    return weighted_avg, preds

```

- Tính sai số tuyệt đối của từng mô hình, sắp xếp mô hình theo độ lệch tăng dần và áp dụng thay đổi theo thứ tự:

```
def update_scores(self, predicted_details, actual_value):
    """Update the confidence score based on feedback"""
    errors = {model_name: abs(predicted_value - actual_value)
              for model_name, predicted_value in predicted_details.items()}

    sorted_models = sorted(errors.items(), key=lambda x: x[1])

    score_changes = [0.001, 0.0005, 0, -0.0005]

    for idx, (model_name, _) in enumerate(sorted_models):
        if idx < len(score_changes):
            change = score_changes[idx]
        else:
            change = score_changes[-1]  # Nu c hn 4 m hnh, p dng mc cui cng

        self.model_scores[model_name] = round(max(0, min(1, self.model_scores[model_name
    ] + change)), 4)

    print("Updated model scores:", self.model_scores)

```

- Xử lý theo loại message
  - Nhận yêu cầu dự đoán

```
            input_data = pd.DataFrame([data["SensorData"]])

            input_data["SoilType"] = encoder_soil.transform(input_data["SoilType"])
            input_data["CropType"] = encoder_crop.transform(input_data["CropType"])
            input_data[["Temparature", "Humidity", "Moisture"]] = scaler.transform(
                input_data[["Temparature", "Humidity", "Moisture"]])

            input_water = input_data[feature_columns]

            predicted_value, all_model_outputs = ensemble_model.predict_best(input_water)

            response = {
                "PredictedWaterUsage": float(predicted_value),
                "Details": {k: float(v) for k, v in all_model_outputs.items()}
            }

            await websocket.send(json.dumps(response))
            print("Sent Prediction:", response)

```

  - Nhận phản hồi thực tế

```
            predicted_details = data["PredictedDetails"]
            actual_value = data["ActualWaterUsage"]

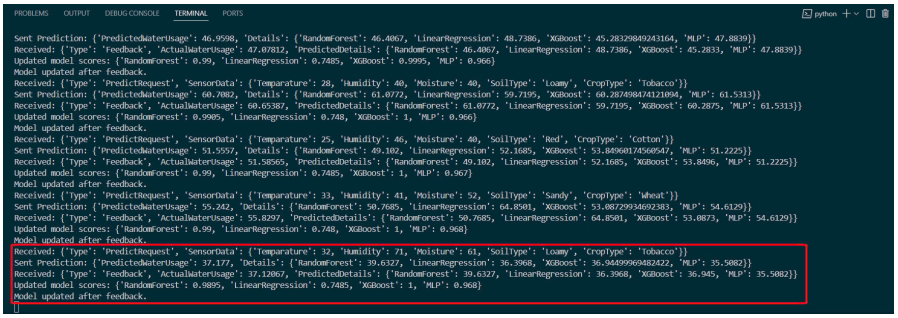
            ensemble_model.update_scores(predicted_details, actual_value)
```

- Lưu lại tệp ensemble_model.pkl

```
            joblib.dump(ensemble_model, "ensemble_model.pkl")
            print("Model updated after feedback.")

            await websocket.send(json.dumps({"Status": "Feedback received and model updated.
"}))
```

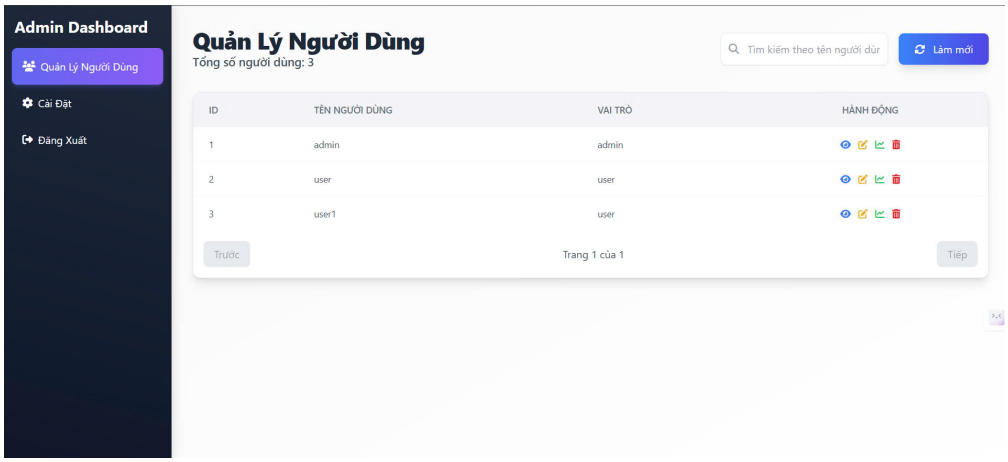
- Kết quả dự đoán và tối ưu hóa lượng nước tưới
  - Dòng đầu tiên là dữ liệu cần dự đoán type = PredictRequest
  - Dòng thứ hai là dữ liệu đã dự đoán, trả về esp, details chứa giá trị dự đoán của 4 mô hình còn PredictedWaterUsage là giá trị trả về cho người dùng, tính dựa trên điểm tin cậy với giá trị dự đoán của 4 mô hình đó
  - Dòng t3 là phản hồi từ esp Type=Feedback, ActualWaterUsage là lượng nước thực tế mà người dùng đã tưới, PredictedDetails là giá trị dự đoán của các mô hình mà server đã gửi (dùng để cập nhật điểm tin cậy cho mô hình)
  - Dòng t4 là điểm tin cậy của các mô hình sau khi cập nhật dữ liệu từ feedback.



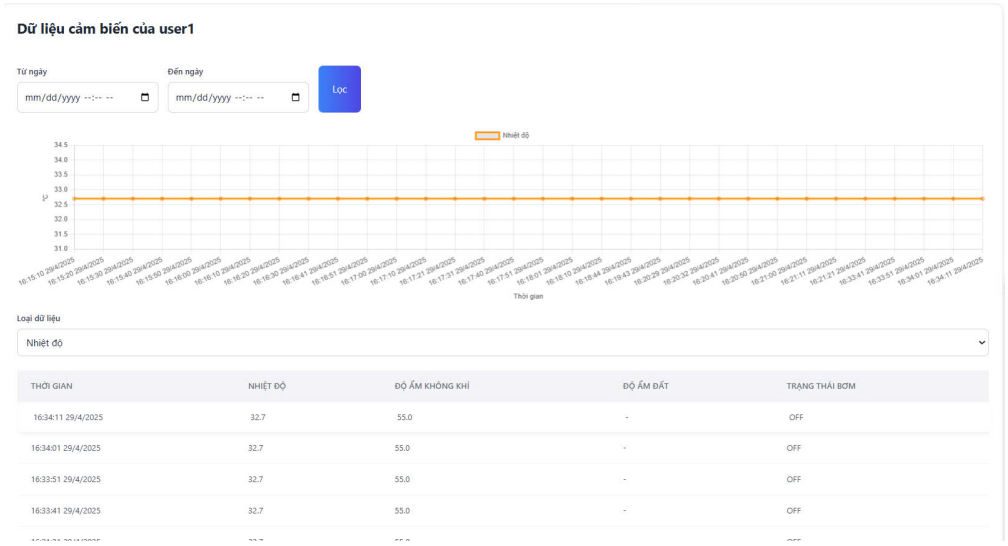
Hình 8: *Kết quả dự đoán và tối ưu hóa lượng nước tưới*

3.6 Dashboard

Trang chủ quản lý của admin:



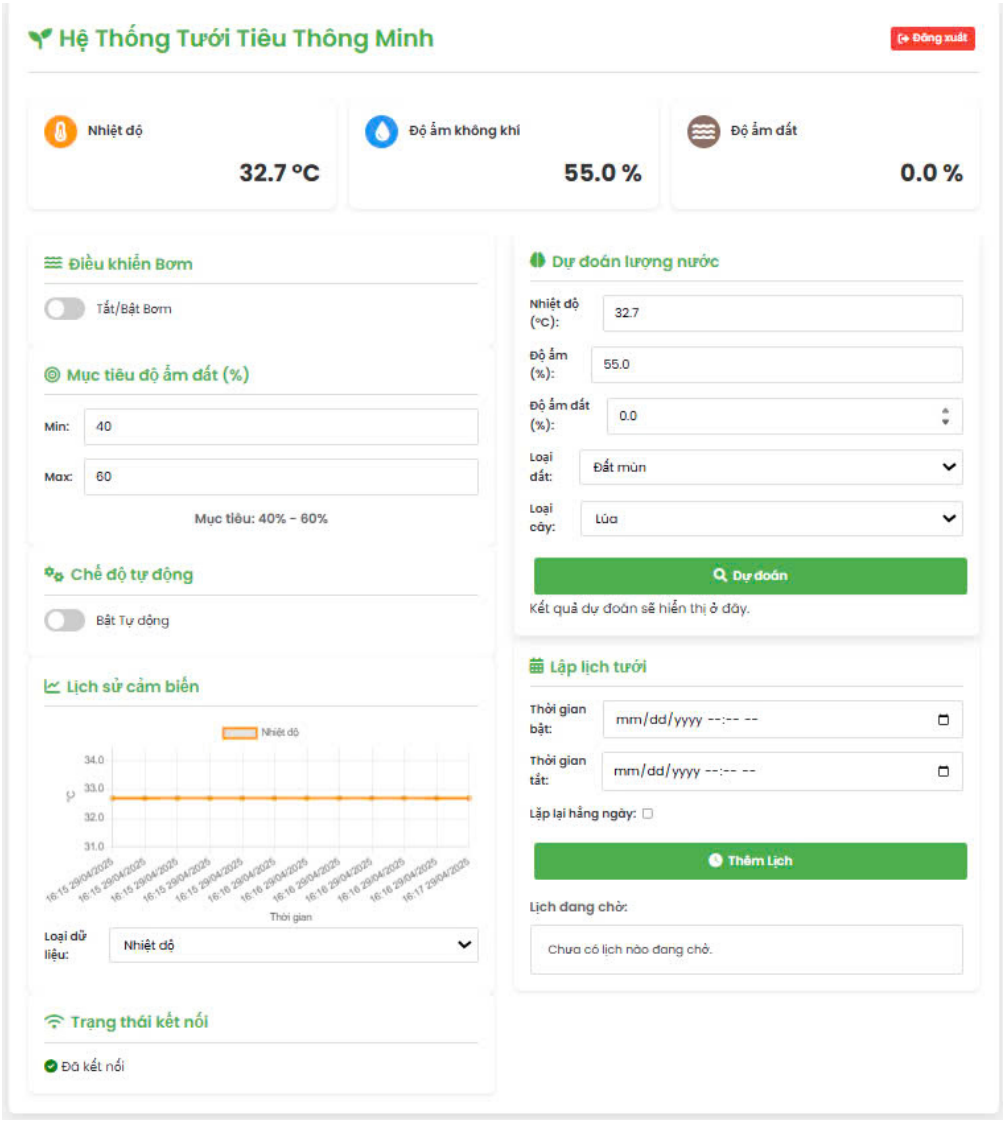
Hình 9: *Admin Dashboard*



Hình 10: *Admin Dashboard*

3.7 Web

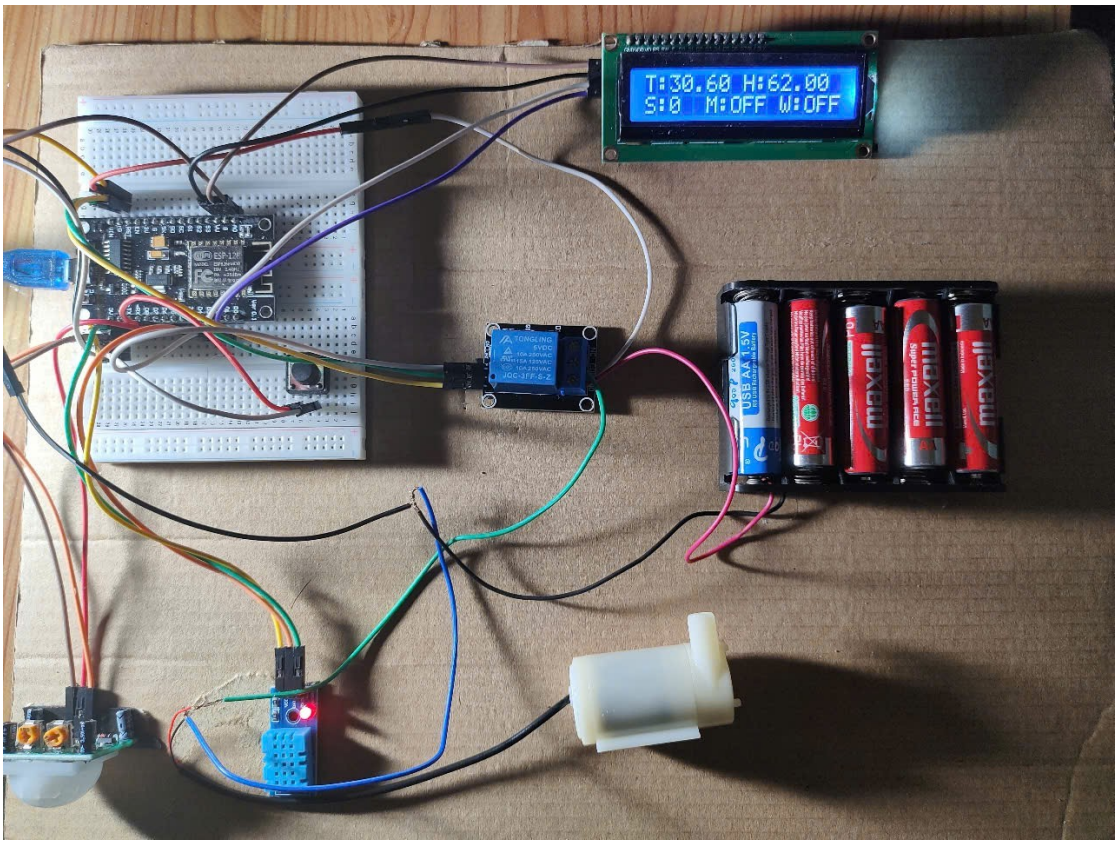
Người dùng có thể theo dõi nhiệt độ, độ ẩm đất, độ ẩm không khí, điều khiển bơm, bật/tắt bơm nước từ xa chỉ với vài cú nhấp chuột trên trang web. Người dùng có thể lập lịch tưới trong khoảng thời gian theo ý muốn của mình. Bên cạnh đó, người dùng có thể dự đoán lượng nước cần tưới và tối ưu lượng nước để tiết kiệm.



Hình 11: *Giao diện*

4 Hình ảnh thực tế

Các thông số của môi trường và hệ thống đều được hiển thị rõ ràng và trực quan trên màn hình LCD 16x2 giúp cho người dùng dễ dàng theo dõi. Bên cạnh đó hệ thống cũng có ưu điểm là rất tiết kiệm năng lượng khi sử dụng nguồn rất thấp, và ngay cả khi mất điện hệ thống vẫn có thể sử dụng pin dự phòng.



Hình 12: *Hình ảnh thực tế*

5 Kết quả thực nghiệm và đánh giá

5.1 Kết quả

- Video demo.

5.2 Đánh giá hiệu quả

Sau quá trình thiết kế và kiểm thử trên nền tảng Adafruit IO, hệ thống tưới tiêu thông minh cho cây trồng do nhóm thực hiện đã cho thấy nhiều hiệu quả rõ

rệt, cả về mặt kỹ thuật lẫn thực tiễn ứng dụng. Các yếu tố được đánh giá bao gồm khả năng tiết kiệm tài nguyên, độ chính xác trong vận hành, tính tự động hóa, khả năng mở rộng và mức độ thân thiện với người dùng.

Thứ nhất, Tiết kiệm nước hiệu quả
Nhờ vào cơ chế thu thập dữ liệu từ cảm biến độ ẩm đất theo thời gian thực, hệ thống chỉ kích hoạt tưới khi thật sự cần thiết – tức là khi độ ẩm đất xuống dưới ngưỡng định trước. Điều này giúp tránh lãng phí nước do tưới quá mức, đồng thời hạn chế hiện tượng úng rễ hoặc sâu bệnh phát sinh. So với hình thức tưới truyền thống (dùng tay hoặc hệ thống tưới hẹn giờ cố định không dựa trên điều kiện môi trường), hệ thống đã giúp tiết kiệm từ 20% đến 30% lượng nước tiêu thụ tùy theo điều kiện thời tiết. Đây là một chỉ số đáng khích lệ, đặc biệt trong bối cảnh biến đổi khí hậu và khan hiếm nguồn nước ngày càng trở nên nghiêm trọng.

Thứ hai, độ chính xác và khả năng tối ưu lượng nước tưới
Dựa trên các ngưỡng độ ẩm đất thu thập được, hệ thống có khả năng ước lượng và điều chỉnh thời gian tưới tương ứng, từ đó tối ưu hóa lượng nước cần thiết cho cây trồng. Tính năng này được tích hợp trong phần mềm thông qua các điều kiện logic đơn giản, kết hợp cùng lịch sử dữ liệu cảm biến để cải thiện chất lượng tưới theo thời gian. Ngoài ra, nhóm cũng xây dựng sơ bộ một hệ thống lập lịch tưới linh hoạt, cho phép người dùng thiết lập khung giờ tưới trong ngày.

Thứ ba, tự động hóa và giảm công sức lao động
Toàn bộ quá trình vận hành của hệ thống đều diễn ra tự động: từ thu thập dữ liệu, xử lý logic tưới, đến cập nhật và hiển thị thông tin lên nền tảng Adafruit IO. Nhờ đó, người dùng không cần thường xuyên can thiệp, đặc biệt hữu ích với người lớn tuổi hoặc khi phải chăm sóc nhiều cây cùng lúc.
So với việc tưới thủ công hằng ngày, hệ thống giúp tiết kiệm đáng kể thời gian và công sức, đồng thời đảm bảo cây vẫn được tưới đúng lúc, đúng lượng nước.

Bên cạnh đó, khả năng giám sát và điều khiển từ xa
Việc tích hợp Adafruit IO giúp hệ thống có khả năng giám sát trạng thái cây trồng từ xa qua điện thoại hoặc máy tính. Người dùng có thể theo dõi các thông số như độ ẩm đất, nhiệt độ, độ ẩm không khí theo thời gian thực.
Đặc biệt, hệ thống còn cho phép điều khiển tưới từ xa bằng cách nhấn nút điều khiển trên dashboard – điều này rất hữu ích khi cần tưới gấp hoặc muốn thử nghiệm hệ thống mà không cần tiếp xúc trực tiếp với phần cứng.
Cuối cùng, khả năng mở rộng và cải tiến
Cấu trúc phần cứng và phần mềm của hệ thống đều được thiết kế mở và dễ mở rộng. Trong tương lai, hệ thống có thể tích hợp thêm các loại cảm biến khác như cảm biến mưa, ánh sáng, pH, hoặc tích hợp AI để phân tích hình ảnh cây trồng, phát hiện bệnh, tự động điều chỉnh chế độ tưới phù hợp.

6 Lời cảm ơn

Nhóm chúng em xin gửi lời cảm ơn chân thành đến Quý Thầy/Cô Khoa khoa học và kỹ thuật máy tính, đặc biệt là Thầy Nguyễn Quang Hùng, người đã tận tình hướng dẫn, hỗ trợ và tạo điều kiện thuận lợi để nhóm thực hiện thành công đồ án với đề tài "Hệ thống tưới tiêu cho cây trồng".

Trong suốt quá trình thực hiện đồ án, chúng em đã có cơ hội vận dụng những kiến thức lý thuyết đã học vào thực tế, đồng thời rèn luyện kỹ năng làm việc nhóm, nghiên cứu, thiết kế hệ thống, cũng như tiếp cận và sử dụng các công nghệ hiện đại như vi điều khiển ESP32, cảm biến môi trường và nền tảng Adafruit IO. Đây là những trải nghiệm quý báu giúp chúng em nâng cao năng lực chuyên môn và tư duy kỹ thuật.

Chúng em cũng xin chân thành cảm ơn bạn bè đã góp ý và hỗ trợ nhóm trong quá trình tìm hiểu, mô phỏng và hoàn thiện hệ thống.

Mặc dù đã nỗ lực hết mình, nhưng do thời gian và kinh nghiệm còn hạn chế, đồ án không tránh khỏi những thiếu sót. Nhóm rất mong nhận được sự góp ý từ Thầy để hoàn thiện hơn trong những lần nghiên cứu tiếp theo.

Trân trọng cảm ơn!

TÀI LIỆU THAM KHẢO

[1] Lumo.ag. (n.d.). What are Smart Irrigation Systems and How Do They Work?. Truy cập từ https://lumo.ag/what-are-smart-irrigation-systems-and-how-do-they-work/.

[2] Sri Tu Hobby. (2023). Smart Irrigation System Using NodeMCU and Blynk. Truy cập từ https://srituhobby.com/smart-irrigation-system-using-nodemcu-and-blynk/